Lab 3: Component Modelling and Architectural Pattern Selection

Self-Service Coffee Kiosk System | Aaryan Dubey, PES2UG24CS016, Sec A

Scenario: Kiosk for ordering Espresso / Americano / Latte in Small or Large size, credit-card payment only, receipt printing, touch-screen UI, and local menu and price storage.

1. Architectural Style Analysis

I compared the three styles against this kiosk, which is a single-machine system with fixed requirements and hardware attached (touch screen, printer):

Layered	Microservices	Client-Server
Simple layers: UI, logic, data. Easy to build and test on one kiosk.	Separate deployable services. Too much ops work for one kiosk.	Central server + many clients. Good for chains, not one standalone kiosk.
Con: layers are tightly linked, but that is fine here since scale is small.	Con: network delay, data consistency issues, no real benefit here.	Con: single point of failure and network dependency for every order.

2. Architecture Selection

We chose Layered Architecture for the Self-Service Coffee Kiosk System.

- **Architectural choice:** Layered Architecture with three layers: Presentation (Touch Screen UI), Business Logic (Order Manager, Payment Service), and Data and Hardware (Menu Repository, Receipt Printer).
- **Reason 1 - fits the problem:** The kiosk does a fixed, simple job on one machine. Layers keep UI, ordering logic, and storage separate, so it is easy to understand and change, for example updating prices without touching the UI code.
- **Reason 2 - avoids over-engineering:** Microservices would add deployment and network complexity for no reason, and Client-Server would need an always-on server. With only 3 coffees, 2 sizes, and card-only payment, a single layered application is enough and more reliable.
- **Security advantage:** All validation goes through the Order Manager and Payment Service. The UI never handles raw card data directly, and menu data is read-only from the local repository, so the attack surface stays small and card details stay isolated in one component.
- **Performance benefit:** Everything runs locally with direct in-process calls, no extra network hops. Touch input, price lookup, and receipt printing stay fast even during rush hours, which matters in a busy cafe.

3. Components and Interfaces (see attached diagram)

Components (5): 1. Touch Screen UI, 2. Order Manager, 3. Payment Service, 4. Menu Repository, 5. Receipt Printer.

Interfaces (4, ball = provided, socket = required): IOrderService (UI requires, Order Manager provides, API call); IPaymentProcessing (Order Manager requires, Payment Service provides, API call); IMenuData (Order Manager requires, Menu Repository provides, DB query); IPrintReceipt (Order Manager requires, Receipt Printer provides, USB / hardware call).

Note: Diagram uses plain UML notation with <<component>> stereotype and assembly connectors, grouped by layer. Exported as PNG with this document.